



AWS Lambda durable functionsでBacklog課題の完了サ
マリーを自動生成し、Slackで承認するフローを構築してみた

2026/03/25

クラスメソッド株式会社 リテールアプリ共創部

高垣 龍平

自己紹介



- 所属
 - クラスメソッド株式会社
 - リテールアプリ共創部 マッハチーム
 - 2024年度 新卒入社
- 名前
 - 高垣龍平
 - X:るおん (@ruonp24)

経歴

classmethod

- 大阪大学
 - 文系（中国語専攻）
- 学生時代：アパレル系自社開発企業（インターン）
 - toC向けWebアプリ
- 2024年4月～：クラスメソッド株式会社
 - Webアプリ新規開発・運用保守
 - LINEミニアプリ開発

技術スタック

- フロントエンド（React/Next.js）
- バックエンド（RoR/Lambda/Express/Next.js）
- インフラ（AWS）
- LINE

実績

- LINE API Expert
- 2025 Japan AWS Jr. Champions



1. 背景・モチベーション
2. デモ
3. なぜ durable functions なのか
4. アーキテクチャ全体像
5. 処理フローの詳細（ソースコード解説）
6. durable functions の重要な概念
7. まとめ

みなさんは顧客とのコミュニケーションツールに何を使っていますか？⁴

よく使われるツール

- Slack / Microsoft Teams / Chatwork
- メール
- Backlog / Jira / Redmine などの課題管理ツール

私たちのチームではBacklogを使っています

→ 課題管理やドキュメント管理等をBacklog上で実施

背景・モチベーション

課題の結論や決定事項がどこにあるかわからない

よくある状況

- コメントが大量にあり、最終的な結論が埋もれている
- 後からBacklogを見返したときに「何が決まったのか」わからない
- 新しいメンバーが背景を理解するのに時間がかかる
- AIに過去のやり取りを読み込ませる際にコンテキストを大量に消費してしまう

課題が完了したら「完了サマリー」を投稿する

→ 後からBacklogを見たときに最初に読むべきところが一目でわかる

完了サマリーの構成

- 課題の内容 - 何についての課題だったか
- 対応内容・経緯 - どんな議論・対応が行われたか
- 課題の結論 - どう解決・完了されたか
- 備考・補足事項 - 追加で記録すべき情報



CM_高垣龍平

2025/12/08 16:25:58

○ 状態: 処理済み → 完了



CM_高垣龍平

2025/12/09 16:22:31

完了サマリー

課題の内容

「商品一覧画面の表示順」について、一覧の並び順に関する仕様を協議。

課題の結論

商品の並び順は以下の降順（最新順）で決定：

1. **最終更新日時**（商品情報が更新された日時）
2. もし上記がなければ、**登録日時**

※商品を「編集」した日 = 最終更新日時として扱う

備考・補足事項

- 関連PBI：商品一覧の表示順を実装する (#26)
- 1stフェーズではカテゴリによるフィルタリングは対象外

これまでの手動フロー

1. Backlog Exporterで課題をローカルにエクスポート
2. Claude Codeのカスタムスラッシュコマンドでサマリー生成
3. 生成されたサマリーをBacklogにコピー

課題

- 毎回手動で3ステップ踏む必要がある
- エクスポート → 生成 → コピーの繰り返し
- 完了した課題が多いと作業が積み上がる

課題が完了したら自動でAIがサマリーを生成し、Slackで承認フローを経てからBacklogにコメントを投稿する

ポイント

- AIが生成したサマリーを**そのまま投稿しない**
- **人間が確認してから**投稿する（承認フロー）
- 完全自動ではなく**Human-in-the-loop**

<https://github.com/takagakiryuheiCM/backlog-completion-notifier>

デモ

デモ：3ステップで完了

12

Step 1

Backlogで課題を完了にする

課題ステータスを「完了」に変更するだけ

Step 2

Slackに承認リクエストが届く

AIが生成したサマリーと「承認して投稿」「却下」ボタンが表示される

Step 3

承認するとBacklogに投稿

「承認して投稿」をクリックすると、Backlogの課題にコメントが自動投稿される

手動作業はゼロ。Backlogで完了にする → Slackで承認するだけ

なぜ durable functions なのか

このシステムの肝はSlackでの人間による承認フロー

通常のLambdaの制約

- 最大タイムアウトは15分
- 人間の承認を数時間～数日待つことがある
- ポーリングで待機するとコンピュータ料金が膨大に

Step Functionsという選択肢もあるが...

- Callback パターンは実装可能だが、状態管理が複雑
- JSONベースのASL (Amazon States Language) で記述が必要
- コードの見通しが悪くなりがち

re:Invent 2025で発表された新機能

最大1年間実行可能

Lambdaの15分制限を超えた**長時間ワークフロー**を構築

待機中コンピューティング料金なし

承認待ちの間は**コストゼロ**。必要なときだけ課金

チェックポイント/リプレイ

障害時も自動復旧。完了済みステップは**スキップ**

通常のコードで記述

Step FunctionsのASLではなく **普通のTypeScript**で書ける

目次

Lambda Durable Functionsとは

主な特徴

想定されるユースケース

チェックポイント・リプレイの仕組み

動作の流れ

Durable Execution SDK

やってみた

前提条件

1. Lambda関数の作成

2. コードの修正とデプロイ

3. 動作確認

注意点・制限事項

現在の制限

最大1年間実行可能な正確な意味

状態保持に関する注意点

おわりに

参考

AWS
re:Invent 2025

やってみた

Just Try

📌 注目の記事

[新機能] 最大1年間実行可能なAWS Lambda Durable Functionsがリリースされたので、公式のサンプルコードを試してみた！ #AWSreInvent

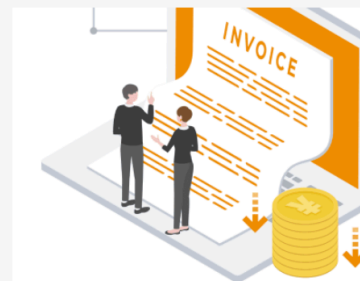
AWS re:Invent 2025

#AWS Lambda

#AWS



Ruon - Ryuhei Takagaki



クラスメソッドの
**AWS請求
代行サービス**で
コスト削減を実現！



請求代行サービスを見る

まずは資料請求だけでも

classmethod

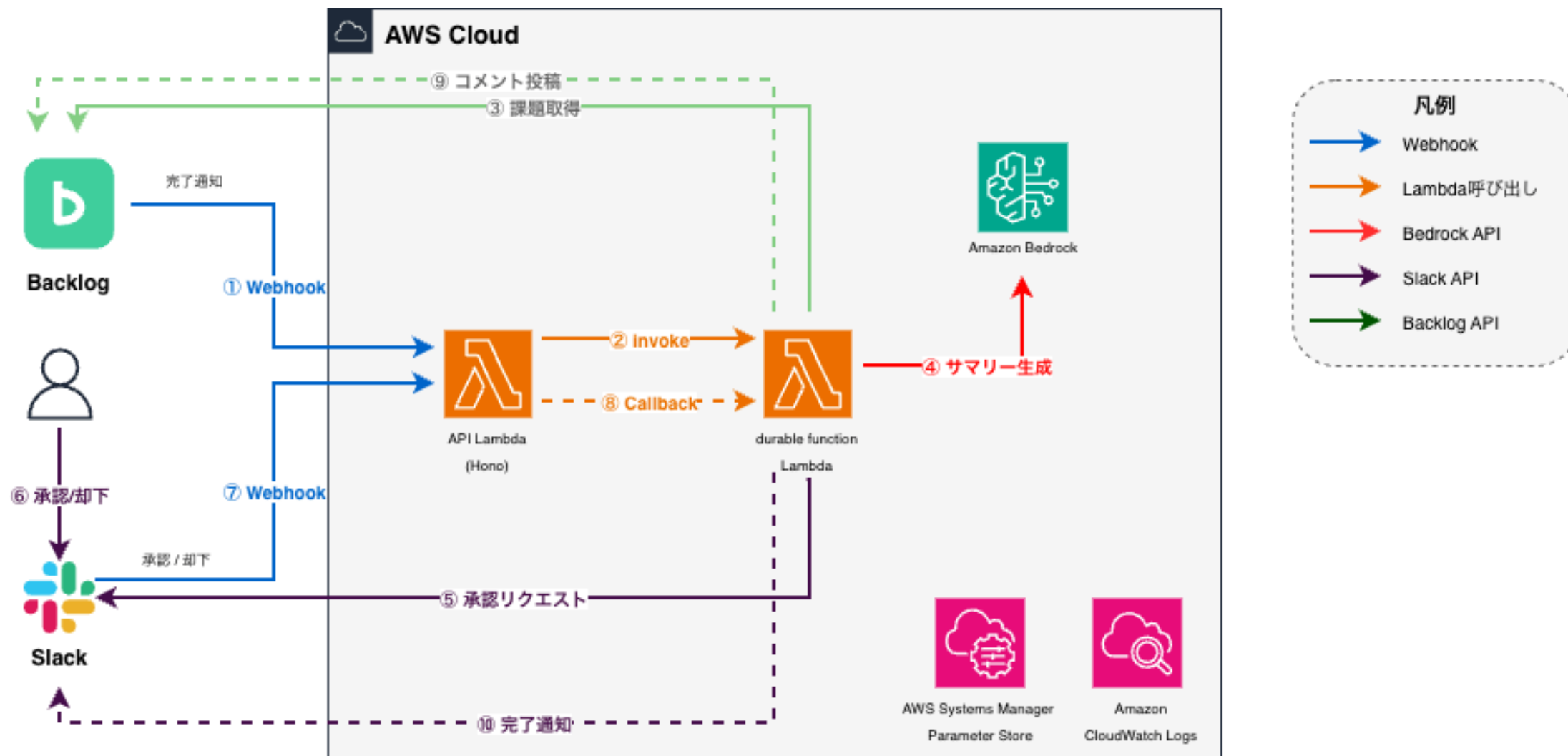
EVENTS



プリミティブ	説明
<code>context.step()</code>	チェックポイント付きのビジネスロジック実行。リプレイ時はスキップ
<code>context.wait()</code>	指定期間の待機。待機中はコンピュータ料金なし
<code>context.createCallback()</code>	外部システムからの入力を待機（人間の承認フローなど）
<code>context.parallel()</code>	複数の操作を並列実行
<code>context.map()</code>	配列の各要素に対して操作を実行
<code>context.invoke()</code>	他のLambda関数を呼び出し

今回のユースケースでは `step()` と `createCallback()` が重要

アーキテクチャ全体像



技術	用途
AWS Lambda	API用とdurable functions用の2つ
Amazon Bedrock (Claude Opus 4.5)	完了サマリーの生成
AWS CDK	インフラのコード管理 (v2.232.0~)
Hono	API Lambdaのルーティング (モノリシック構成)
inversify	依存性注入 (DIコンテナ)
Zod	Webhookリクエストのバリデーション
Backlog API	課題情報の取得・コメント投稿
Slack API	承認リクエスト送信・結果通知
SSM Parameter Store	秘匿情報 (APIキー、Bot Token) の管理

```
backlog-completion-notifier/
├── infra/
│   ├── bin/infra.ts
│   ├── lib/stack/server-stack.ts
│   ├── lib/util/ssm.ts
│   └── config.ts / config-type.ts
├── server/
│   └── src/
│       ├── handler/
│       │   ├── api/
│       │   │   ├── handler.ts / app.ts
│       │   │   ├── route/webhook/
│       │   │   ├── schema/
│       │   │   └── middleware/
│       │   └── durable/handler.ts
│       ├── use-case/
│       ├── domain/
│       ├── infrastructure/
│       └── di-container/
```

CDKインフラコード
CDKエントリポイント
Lambda定義
SSMパラメータ取得
設定
サーバーコード

API Lambda (Hono)
Webhookハンドラー
Zodスキーマ
Slack payload validator
Durable Function
ユースケース層
ドメイン層 (型・テンプレート)
インフラ層 (外部API実装)
依存性注入 (inversify)

Handler層

- Webhookの受信
- リクエストのバリデーション (Zod)
- UseCaseの呼び出し
- レスポンスの返却

UseCase層

- ビジネスロジックの実行
- 各サービスの組み合わせ
- Durableのワークフロー管理

Domain層

- 型定義・インターフェース
- テンプレート (プロンプト/Slack/Backlog)
- ビジネスルール

Infrastructure層

- 外部API実装 (Backlog/Slack/Bedrock)
- DurableFunctionClient
- Logger

API Lambda (backlog-completion-api)

- Honoでルーティング
- Backlog WebhookとSlack Webhookを受信
- `/webhook/backlog` → Durable起動
- `/webhook/slack` → コールバック送信
- Function URLで公開

Durable Lambda (backlog-completion-durable)

- `withDurableExecution`でラップ
- 課題情報取得 → サマリー生成 → 承認待機
- `durableConfig` で待機時間を設定
- 承認後にBacklogへコメント投稿
- Bedrockを呼び出す権限あり

Durable FunctionはFunction URLやAPI Gatewayからの呼び出し自体は可能

*"Durable Lambda functions support **the same invocation methods** as standard Lambda functions"*

— [AWS公式ドキュメント](#)

しかし、コールバックの仕組みが分離を必要とする

待機中のDurable Functionを再開するには、**Lambda API**経由で

`SendDurableExecutionCallbackSuccess` を呼ぶ必要がある。これはDurable Function自身への新しいinvocationではなく**AWS API**コール

各invocationは独立したdurable executionを生成する

Slackのwebhookが同じDurable Functionを叩いても、既存の待機中のexecutionには届かない。
新しいdurable executionが始まるだけ

そのため、以下の構成が必要

Slackボタンクリック

- API Lambda（通常のLambda）がwebhookを受信
- SendDurableExecutionCallbackSuccessCommand を実行（AWS API）
- 待機中のDurable Functionのexecutionが再開

コールバックを受け取り、Lambda APIを呼ぶ「仲介役」が必要 → それがAPI Lambda

```
const durableFunction = new nodejs.NodejsFunction(this, "DurableFunction", {
  functionName: durableFunctionName,
  runtime: lambda.Runtime.NODEJS_22_X,
  entry: path.join(serverSrcPath, "handler/durable/handler.ts"),
  handler: "handler",
  timeout: cdk.Duration.seconds(30),
  memorySize: 1024,
  architecture: lambda.Architecture.ARM_64,
  environment: {
    ...commonEnv,
    BACKLOG_API_KEY, BACKLOG_SPACE_ID: config.backlogSpaceId,
    SLACK_BOT_TOKEN, SLACK_CHANNEL_ID: config.slackChannelId,
  },
  // Durable Functionsの設定
  durableConfig: {
    executionTimeout: cdk.Duration.days(1), // 最大24時間待機可能
    retentionPeriod: cdk.Duration.days(7), // 実行履歴を7日間保持
  },
});
```

ポイント: **durableConfig** を設定するだけでDurable Functionsが有効化される (CDK v2.232.0~)

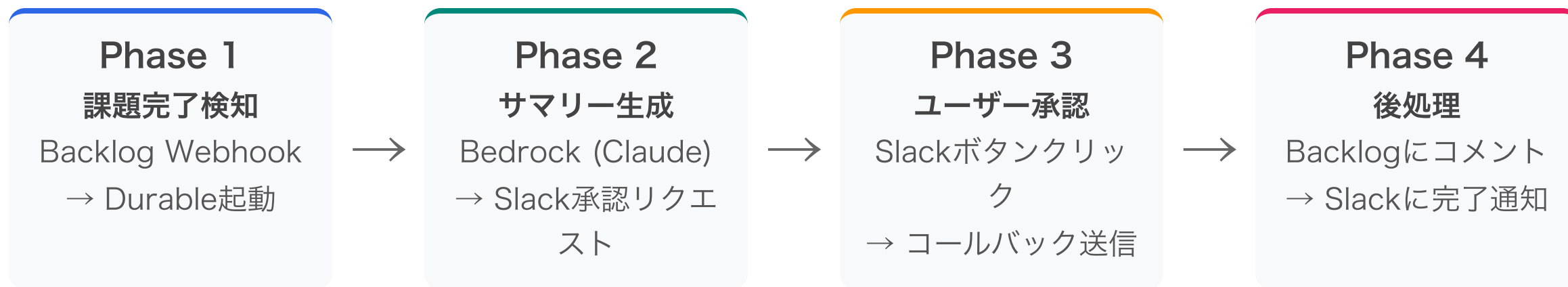
```
// Durable FunctionにBedrock呼び出し権限を付与
durableFunction.addToRolePolicy(
    new iam.PolicyStatement({
        actions: ["bedrock:InvokeModel"],
        resources: ["*"],
    })
);

// API LambdaがDurable Functionを呼び出す権限
durableFunction.grantInvoke(apiFunction);

// API LambdaがDurable Functionのコールバックを送信する権限
apiFunction.addToRolePolicy(
    new iam.PolicyStatement({
        actions: [
            "lambda:SendDurableExecutionCallbackSuccess",
            "lambda:SendDurableExecutionCallbackFailure",
        ],
        resources: [durableFunction.functionArn, `${durableFunction.functionArn}:*`],
    })
);
```

SendDurableExecutionCallbackSuccess/Failure はdurable functions専用のIAMアクション

処理フローの詳細



Phase 2 → Phase 3 の間で**最大24時間の待機**が発生（コンピュータ料金なし）

Backlog Webhookを受信し、課題が完了かどうか判定

```
async execute(input: HandleBacklogWebhookInput): Promise<HandleBacklogWebhookOutput> {  
  // 課題更新イベント以外は無視  
  if (input.type !== BacklogEventType.ISSUE_UPDATED) {  
    return { status: "ignored", reason: "Not an issue update event" };  
  }  
  
  // 課題が完了状態でなければ無視  
  if (!isIssueCompleted(input)) {  
    return { status: "ignored", reason: "Issue is not completed" };  
  }  
  
  const issueKey = getIssueKey(input);  
  
  // Durable Functionを非同期起動  
  await this.#durableFunctionClient.invoke({  
    issueKey,  
    projectKey: input.project.projectKey,  
    issueSummary: input.issue.summary,  
    issueDescription: input.issue.description || "",  
  });  
  
  return { status: "invoked", issueKey };  
}
```

InvocationType: "Event" で非同期呼び出し

```
export class LambdaDurableFunctionClient implements DurableFunctionClient {
  async invoke(params: DurableFunctionParams): Promise<void> {
    // Durable Functionはqualified ARN (バージョン指定) が必要
    const command = new InvokeCommand({
      FunctionName: this.#functionName,
      Qualifier: "$LATEST",
      InvocationType: "Event", // 非同期呼び出し
      Payload: JSON.stringify(params),
    });
    await this.#lambdaClient.send(command);
  }
}
```

ポイント

- **Qualifier: "\$LATEST"** が必要 (Durable Functionsの要件)
- **InvocationType: "Event"** で非同期呼び出し (API Lambdaは即座にレスポンスを返す)

withDurableExecution でハンドラーをラップ

```
import { withDurableExecution, DurableContext } from "@aws/durable-execution-sdk-js";

interface DurableFunctionInput {
  issueKey: string;
  projectKey: string;
  issueSummary: string;
  issueDescription: string;
}

const container = registerContainer();

export const handler = withDurableExecution(
  async (event: DurableFunctionInput, context: DurableContext) => {
    const useCase = container.get<ProcessIssueCompletionUseCase>(
      serviceId.PROCESS_ISSUE_COMPLETION_USE_CASE
    );
    return useCase.execute({ issueKey: event.issueKey }, context);
  }
);
```

withDurableExecution でラップすることで **DurableContext** が利用可能に

`context.step()` でチェックポイントを設定しながら処理を進める

```
// Step 1: 課題詳細とコメントを取得
const issueDetails = await context.step("fetch-issue-details", async () => {
  const [issue, comments] = await Promise.all([
    this.#backlogRepository.getIssue(issueKey),
    this.#backlogRepository.getComments(issueKey),
  ]);
  return { issueKey, issueSummary: issue.summary, /* ... */ };
});

// Step 2: 完了サマリーを生成 (Bedrock Claude Opus 4.5)
const summary = await context.step("generate-summary", async () => {
  const prompt = buildCompletionSummaryPrompt({
    issueKey: issueDetails.issueKey,
    issueSummary: issueDetails.issueSummary,
    issueDescription: issueDetails.issueDescription,
    comments: issueDetails.comments,
  });
  return this.#summaryGenerator.generate(prompt);
});
```

Phase 2（続き）：コールバック作成とSlack通知

```
// Step 3: コールバックを作成（最大24時間待機）
const [callbackPromise, callbackId] = await context.createCallback("approval", {
  timeout: { hours: 24 },
});

// Step 4: Slack承認リクエストを送信
await context.step("send-approval-request", async () => {
  const message = buildApprovalMessage({
    channel: this.#slackChannelId,
    issueKey: issueDetails.issueKey,
    issueSummary: issueDetails.issueSummary,
    issueUrl: issueDetails.issueUrl,
    callbackId, // ← ボタンにcallbackIdを埋め込む
  });
  await this.#slackNotifier.postMessage(message);
});

// Step 4.5: サマリーをスレッドに投稿
await context.step("send-summary-thread", async () => {
  await this.#slackNotifier.postMessage({
    channel: this.#slackChannelId,
    text: `*完了サマリー:* \n${summary}`,
    thread_ts: approvalMessageResult.ts,
  });
});

// ここで待機！（コンピュート料金なし）
const result = await callbackPromise;
```

`createCallback()` で生成した `callbackId` を Slackボタンの `value` に埋め込む
→ ユーザーがボタンをクリックしたとき、`callbackId` で待機中のDurable Functionを特定できる

```
// Slackの承認ボタンにcallbackIdを埋め込む
elements: [
  {
    type: "button",
    text: { type: "plain_text", text: "承認して投稿" },
    style: "primary",
    action_id: "approve",
    value: JSON.stringify({ callbackId, approved: true }),
  },
  {
    type: "button",
    text: { type: "plain_text", text: "却下" },
    style: "danger",
    action_id: "reject",
    value: JSON.stringify({ callbackId, approved: false }),
  },
],
```

Slackボタンクリック → コールバック送信

```
async execute(input: HandleSlackWebhookInput): Promise<HandleSlackWebhookOutput> {
  const { callbackId, approved, userName } = input;

  if (approved) {
    // 承認: SendDurableExecutionCallbackSuccessCommand
    await this.#durableFunctionClient.sendCallbackSuccess(callbackId, {
      approved: true,
      approvedBy: userName,
      approvedAt: this.#fetchNow().toISOString(),
    });
  } else {
    // 却下: SendDurableExecutionCallbackFailureCommand
    await this.#durableFunctionClient.sendCallbackFailure(callbackId, {
      rejectedBy: userName,
    });
  }

  return buildApprovalResponse({ approved, userName });
}
```

AWS SDK v3の新コマンドを使用

```
async sendCallbackSuccess(callbackId: string, result: ApprovalResult): Promise<void> {
  const command = new SendDurableExecutionCallbackSuccessCommand({
    CallbackId: callbackId,
    Result: new TextEncoder().encode(JSON.stringify(result)),
  });
  await this.#lambdaClient.send(command);
}

async sendCallbackFailure(callbackId: string, rejection: RejectionInfo): Promise<void> {
  const command = new SendDurableExecutionCallbackFailureCommand({
    CallbackId: callbackId,
    Error: {
      ErrorType: "REJECTED",
      ErrorMessage: `Rejected by ${rejection.rejectedBy}`,
    },
  });
  await this.#lambdaClient.send(command);
}
```

SendDurableExecutionCallbackSuccessCommand / **FailureCommand** は最新のAWS SDK v3で追加

コールバック解決後、Durable Functionがリプレイされる

```
// callbackPromiseが解決された後の処理
try {
  const result = await callbackPromise;
  approval = JSON.parse(result) as CallbackResult;
} catch {
  // 却下またはタイムアウト → Slackに却下通知
  await context.step("send-rejection-notification", async () => {
    await this.#slackNotifier.postMessage(/* 却下通知 */);
  });
  return { issueKey, status: "rejected", summary };
}

// 承認時の処理
if (approval.approved) {
  // Step 5: Backlogにコメント投稿
  await context.step("post-backlog-comment", async () => {
    const comment = buildBacklogComment({ summary });
    await this.#backlogRepository.addComment(issueKey, comment);
  });

  // Step 6: Slackに完了通知
  await context.step("send-completion-notification", async () => {
    await this.#slackNotifier.postMessage(/* 完了通知 */);
  });
}
```

durable functions の重要な概念

リプレイの流れ

1. 関数が実行され、各 `step()` 完了時に**チェックポイント**が保存される
2. 障害発生 or `wait()` / `callback` 完了時に**関数を最初から再実行**
3. 完了済み `step()` は**スキップ**され、保存された結果を使用
4. 未完了の処理から**再開**

メリット

- **障害耐性** - 処理中にエラーが発生しても、チェックポイントから再開
- **長時間実行** - 待機中はコンピュータ料金が発生しない
- **冪等性** - リプレイ時に同じ処理が重複実行されない

決定論的なコードが必要

`context.step()` 外でランダムな値や `Date.now()` を使うと、リプレイ時に結果が変わってしまう

stepの返り値で状態を引き継ぐ

グローバル変数や `return` していない値はリプレイ時にリセットされる。次のステップに引き継ぎたい値は必ず `return`

15分のタイムアウトは変わらない

「最大1年間実行可能」は待機を挟んだワークフロー全体の話。各stepの実行自体は15分以内

Callback の try-catch

`sendCallbackFailure` が送信された場合やタイムアウトの場合は Promise が `reject` される。try-catchで却下処理を行う

まとめ

durable functionsで人間の承認フローを実現

Lambda内で最大1年間待機可能。待機中はコンピュータ料金なし

callbackIdの受け渡しがシステム連携の鍵

createCallback() → Slackボタンに埋め込み → SendDurableExecutionCallbackSuccess で待機解除

チェックポイント/リプレイで障害耐性を確保

step()で保存したチェックポイントにより、リプレイ時に完了済み処理をスキップ

普通のTypeScriptコードで書ける

Step FunctionsのASLではなく、馴染みのある言語でワークフローを記述可能

- ブログ記事

- <https://dev.classmethod.jp/articles/aws-lambda-durable-functions-backlog-slack/>

- GitHub リポジトリ

- <https://github.com/takagakiryuheiCM/backlog-completion-notifier>

- AWS 公式ドキュメント

- <https://docs.aws.amazon.com/lambda/latest/dg/durable-functions.html>
- <https://docs.aws.amazon.com/lambda/latest/dg/durable-execution-sdk.html>

- Durable Execution SDK

- <https://github.com/aws/aws-durable-execution-sdk-js>

The logo for 'classmethod' is displayed within a black-outlined rectangular frame. The frame has a small notch at the top right corner. The word 'classmethod' is written in a bold, lowercase, sans-serif font.

ご清聴ありがとうございました